Prep: Imports and Data Load

```
[1]: import pandas as pd
     from sklearn.model_selection import train_test_split
     from sklearn.ensemble import RandomForestClassifier
     from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns

     #Load the original dataset
     file_path = "breast-cancer.csv"
     df = pd.read_csv(file_path)
```

Clean: Double check for Missing,Null, and 0 Values in Data.

```
#Check for missing values in the dataset
print("Checking for missing values in the dataset:")
print(df.isnull().sum())

#Check for zero values (if applicable)
print("\nChecking for zero values in the dataset:")
print((df == 0).sum())
```

```
Checking for missing values in the dataset:
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

```
Checking for zero values in the dataset:
id                          0
diagnosis                   0
radius_mean                 0
texture_mean                0
perimeter_mean              0
area_mean                   0
smoothness_mean             0
compactness_mean            0
concavity_mean             13
concave points_mean        13
symmetry_mean               0
fractal_dimension_mean      0
radius_se                   0
texture_se                  0
perimeter_se                0
area_se                     0
smoothness_se               0
compactness_se              0
concavity_se               13
concave points_se          13
symmetry_se                 0
fractal_dimension_se        0
radius_worst                0
texture_worst               0
perimeter_worst             0
area_worst                  0
smoothness_worst            0
compactness_worst           0
concavity_worst            13
concave points_worst       13
symmetry_worst              0
fractal_dimension_worst     0
```

Findings: There are no missing values in the data, but in the headers that reguarding concavity overall there are 13 zeros in each . Since these are real zeros for actual data it should not affect my work.

<u>Data Integrity Check: Drop ID Column because it will affect accuracy of training model.</u>

```
# Drop ID because it is ireelevant to know the annonymous patients used in the sample data
df = df.drop(columns=['id'], errors='ignore')
```

Findings: Before proceeding, I dropped the id column from the dataset. Although it had no predictive value, its presence in earlier parts may have contributed to overfitting. Removing it ensured that the model trained only on meaningful features. After dropping id, there was a slight adjustment in performance metrics but no significant negative impact on model strength.

Step 1: Change Benign and Malignant to 0 and 1

```
#STEP 1
#Ensure the 'diagnosis' column is binary: 0 for Benign ('B'), 1 for Malignant ('M')
df['diagnosis'] = df['diagnosis'].map({'B': 0, 'M': 1})

#Save the cleaned dataframe to a new file
updated_file_path = "breast-cancer-updated.csv"
df.to_csv(updated_file_path, index=False)

print(f"\nThe cleaned dataset has been saved to {updated_file_path}")
```

Findings: When looking at the data to figure out what I needed to use to do comparisons I found that there are two types of breast cancer diagnosis "Benign and Malignant", they were labeled "B" and "M". I decided to choose these two for the binary classifier mapping them to be 0 for "B" and 1 for "M" using. map function.

Step 2: Split the Data for Model Training and Evaluation

```
[16]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np
#STEP 2
# Separate features (X) and labels (y)
X = df.drop(columns=['diagnosis'])  # Features
y = df['diagnosis']  # Labels (0 = Benign, 1 = Malignant)

#Split the data into Training (60%), Validation (20%), and Test (20%) sets
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42)  # 60% training, 40% temp
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)  # 50% of the 40% for validation, 50% for test

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Validation set size: {X_val.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")

Training set size: 341 samples
Validation set size: 114 samples
Test set size: 114 samples
```

```
[17]: #Train a Random Forest Classifier
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

#Predict on Training, Validation, and Test sets
y_train_pred = model.predict(X_train)
y_val_pred = model.predict(X_val)
y_test_pred = model.predict(X_test)
```

```
Training set size: 341 samples
Validation set size: 114 samples
Test set size: 114 samples
```

Findings: Given the large dataset with over 300 samples, I opted to use a Random Forest model instead of performing cross-validation. First, I separated the features and diagnosis labels, where benign cases were labeled as 0 and malignant cases as 1. Next, I split the data into three sets:

training (60%), validation (20%), and test (20%) using scikit-learn's train_test_split function. The training set consisted of 341 samples, while both the validation and test sets contained 114 samples each. I then trained a RandomForestClassifier on the training data and used it to make predictions on all three datasets. This provided a foundation for further model evaluation and performance analysis.

Step 3: Evaluate Model Performance with Confusion Matrices

```
# STEP 3: Calculate Accuracy on Training, Validation, and Test sets
train_accuracy = accuracy_score(y_train, y_train_pred)
val_accuracy = accuracy_score(y_val, y_val_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_accuracy:.4f}")
print(f"Validation Accuracy: {val_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Training Accuracy: 1.0000
Validation Accuracy: 0.9737
Test Accuracy: 0.9649
```

Findings: I evaluated my model's performance by calculating accuracy on the training, validation, and test sets. The training accuracy was 100%, indicating that the model perfectly classified all training data points. The validation accuracy was 97.37%, and the test accuracy was 96.49%, both of which are strong indicators that the model is generalizing well to unseen data. Despite the perfect training accuracy, the small drop in validation and test performance suggests that the model is performing reliably rather than simply memorizing the data. This points to a strong balance between fit and generalization, especially after removing the id column to prevent unintentional overfitting.

4: Model Performance Metrics: Accuracy, Precision, Recall, and F1 Score

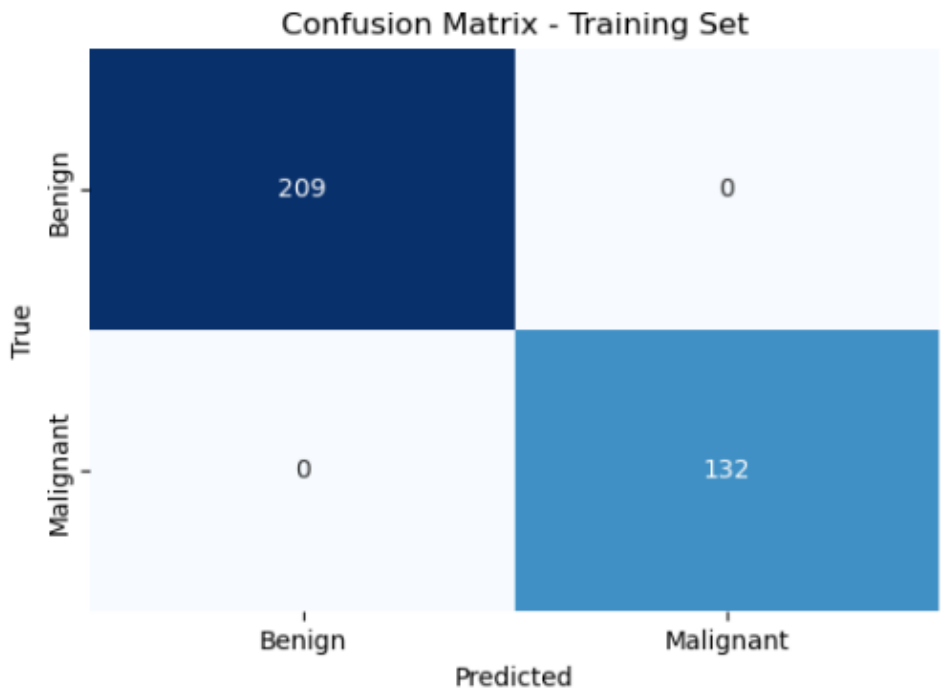
```python
# STEP 4: Calculate Precision, Recall, and F1 Scores for Training, Validation, and Test sets
train_report = classification_report(y_train, y_train_pred, output_dict=True)
val_report = classification_report(y_val, y_val_pred, output_dict=True)
test_report = classification_report(y_test, y_test_pred, output_dict=True)

# Print Classification Reports
print("Training Classification Report:")
print(classification_report(y_train, y_train_pred))
print("Validation Classification Report:")
print(classification_report(y_val, y_val_pred))
print("Test Classification Report:")
print(classification_report(y_test, y_test_pred))

#Create Confusion Matrix after Training, Validation, and Testing
train_cm = confusion_matrix(y_train, y_train_pred)
val_cm = confusion_matrix(y_val, y_val_pred)
test_cm = confusion_matrix(y_test, y_test_pred)

# Plot Confusion Matrices
def plot_confusion_matrix(cm, dataset_name):
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
    plt.title(f'Confusion Matrix - {dataset_name}')
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.show()

# Plot Confusion Matrices for each dataset
plot_confusion_matrix(train_cm, 'Training Set')
plot_confusion_matrix(val_cm, 'Validation Set')
plot_confusion_matrix(test_cm, 'Test Set')
```

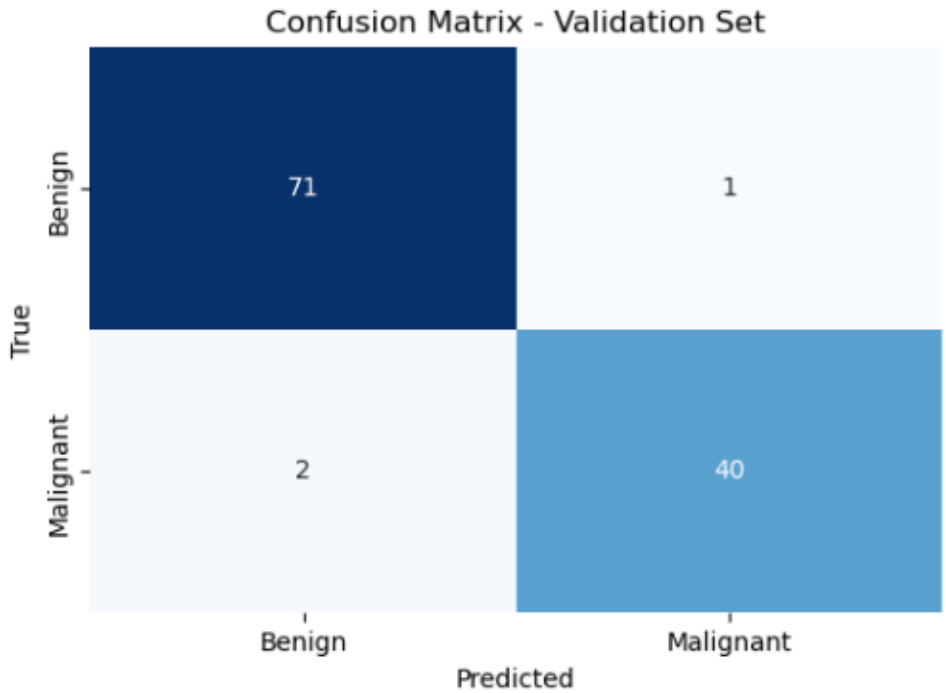
```
Training Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       209
           1       1.00      1.00      1.00       132

    accuracy                           1.00       341
   macro avg       1.00      1.00      1.00       341
weighted avg       1.00      1.00      1.00       341

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        72
           1       0.98      0.95      0.96        42

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

Test Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        76
           1       1.00      0.89      0.94        38

    accuracy                           0.96       114
   macro avg       0.97      0.95      0.96       114
weighted avg       0.97      0.96      0.96       114
```

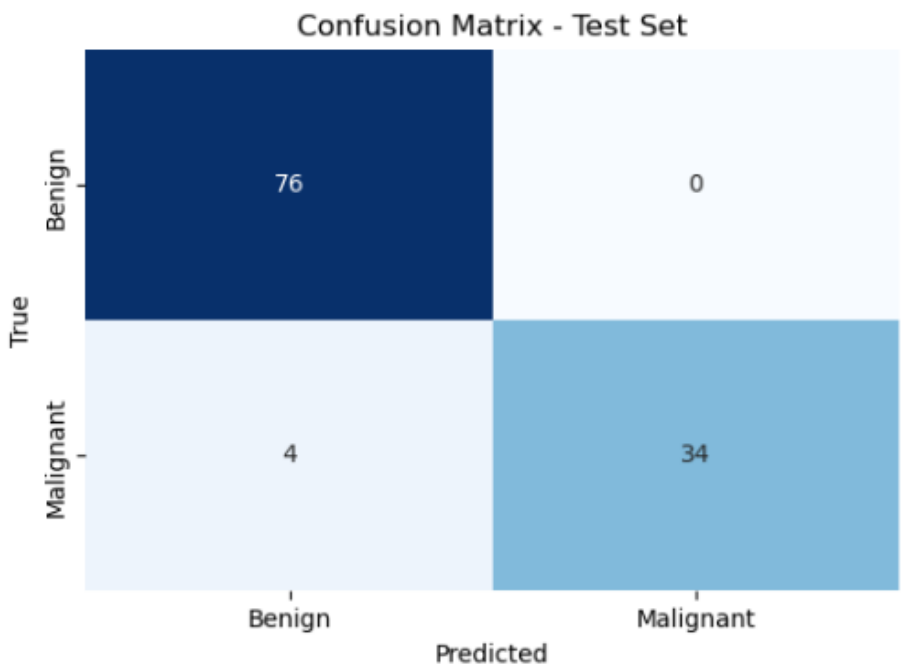
Findings: First, I built a classification model to distinguish between benign and malignant tumors. To thoroughly evaluate its performance, I divided my dataset into training, validation, and test sets. I used the training set to train the model, the validation set to fine-tune parameters, and the test set to assess generalization to unseen data.

I generated classification reports for each set using the classification_report function to calculate precision, recall, and F1-scores. On the training set, the model achieved perfect performance, with 100% precision, recall, and F1-score for both benign (0) and malignant (1) classes. On the validation set, the model maintained excellent results, achieving 0.97 precision and 0.99 recall for benign cases, and 0.98 precision and 0.95 recall for malignant cases, with an overall F1-score of 0.97. On the test set, the model showed strong performance, though with a slight reduction in recall for malignant cases. The precision and recall for benign cases were 0.95 and 1.00 respectively, while malignant cases had 1.00 precision and 0.89 recall. This resulted in an overall F1-score of 0.96.

These results indicate that the model is highly accurate and well-balanced across all sets. However, despite removing the id column, the perfect training performance suggests that the model may still be slightly overfitting. The high validation and test scores indicate strong generalization, but the small gap between training and unseen data suggests that further improvements—such as tuning model complexity, reducing feature count, or using cross-validation—could enhance reliability even more. I also created confusion matrices for each set using seaborn's heatmap to visualize misclassifications more clearly.







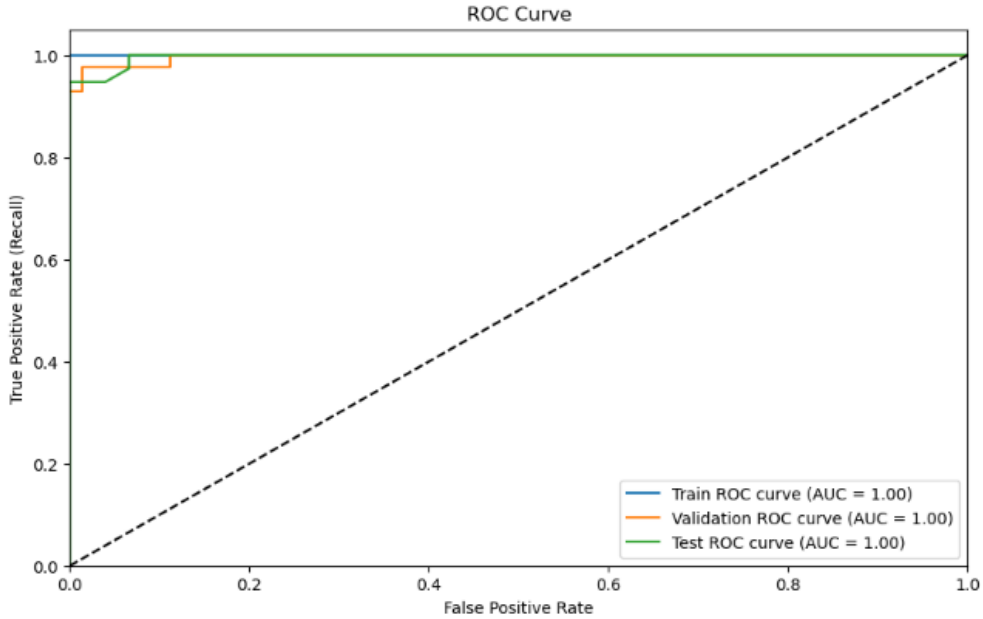
Moving onto the confusion matrices seen above, the model achieved perfect performance on the training set, with no misclassifications. This reinforces the observed 100% training accuracy. The validation set revealed one false positive and two false negatives, indicating a small but acceptable margin of error. On the test set, the model correctly identified all benign cases but misclassified four malignant cases as benign, resulting in four false negatives. This

pattern is especially important in a medical context, where failing to detect malignant tumors can have serious consequences. While the overall performance remains high, the presence of false negatives in both the validation and test sets suggests there is still room for improvement. These findings support the need to continue addressing potential overfitting and to consider strategies such as additional feature selection, hyperparameter tuning, or cross-validation to improve the model's ability to reliably detect malignant cases.

Step 5: Clculate FPR and TNR and plot the ROC curve

[20]:
```
from sklearn.metrics import roc_curve, auc

# STEP 5
# Predict probabilities for the positive class (class 1)
y_train_prob = model.predict_proba(X_train)[:, 1]  # Probabilities for class 1
y_val_prob = model.predict_proba(X_val)[:, 1]      # Probabilities for class 1
y_test_prob = model.predict_proba(X_test)[:, 1]    # Probabilities for class 1

#Calculate ROC curve (FPR, TPR) for each set
train_fpr, train_tpr, _ = roc_curve(y_train, y_train_prob)
val_fpr, val_tpr, _ = roc_curve(y_val, y_val_prob)
test_fpr, test_tpr, _ = roc_curve(y_test, y_test_prob)

# Calculate AUC (Area Under the Curve) for each set
train_auc = auc(train_fpr, train_tpr)
val_auc = auc(val_fpr, val_tpr)
test_auc = auc(test_fpr, test_tpr)

#Calculate FPR and TNR (True Negative Rate) for each set
train_tnr = 1 - train_fpr  # TNR = 1 - FPR
val_tnr = 1 - val_fpr      # TNR = 1 - FPR
test_tnr = 1 - test_fpr    # TNR = 1 - FPR

# Display FPR and TNR for each set
print(f"Training FPR: {train_fpr[1]:.2f}, Training TNR: {train_tnr[1]:.2f}")
print(f"Validation FPR: {val_fpr[1]:.2f}, Validation TNR: {val_tnr[1]:.2f}")
print(f"Test FPR: {test_fpr[1]:.2f}, Test TNR: {test_tnr[1]:.2f}")

#Plot the ROC curves for Training, Validation, and Test sets
plt.figure(figsize=(10, 6))
plt.plot(train_fpr, train_tpr, label=f'Train ROC curve (AUC = {train_auc:.2f})')
plt.plot(val_fpr, val_tpr, label=f'Validation ROC curve (AUC = {val_auc:.2f})')
plt.plot(test_fpr, test_tpr, label=f'Test ROC curve (AUC = {test_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')  # Random classifier (diagonal line)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.show()
```

```
Training FPR: 0.00, Training TNR: 1.00
Validation FPR: 0.00, Validation TNR: 1.00
Test FPR: 0.00, Test TNR: 1.00
```



Findings: To evaluate my binary classification model, I implemented ROC curve analysis and calculated the Area Under the Curve (AUC). I began by extracting predicted probabilities for the positive class across my training, validation, and test datasets. Using these, I computed the False Positive Rate (FPR) and True Positive Rate (TPR) at various thresholds, which allowed me to generate the ROC curve data for each dataset. I then calculated the AUC for each dataset as a summary performance metric and printed the FPR and True Negative Rate (TNR) at the default threshold.

Interestingly, the ROC curves for all three sets were still perfectly shaped, rising steeply along the y-axis and flattening across the top, indicating near-ideal classification probability separation. The AUC scores remained at 1.0 for training, validation, and test sets. This may seem inconsistent with the confusion matrix, where the test set showed four false negatives. However, this can be explained by the nature of ROC curves: AUC is based on the model's predicted probabilities across all thresholds, not just the 0.5 cutoff used in the classification report. In this case, the model likely ranked all malignant samples with higher probability than benign ones, even if a few were misclassified at the default threshold. This results in a perfect AUC despite some misclassifications. While this highlights the strength of the model's probability estimates, the presence of test set false negatives still suggests some overfitting, and further tuning may be needed to increase reliability under real-world threshold settings.